

Student notes

How would we know that a requirement has been satisfied?

Student notes for lecture 2. Every scene shows a situation, asks you to predict, reveals the mechanism, changes one condition, compares the outcomes and names the principle. The pages that follow keep the selected states of each scene with their captions, and the register page collects the requirements written this week.

Scenes in this lecture

#	SCENE (STUDENT HEADING)	ROLE	DEEP LINK
01	One sentence, three systems	Anchor	week-02.html#/ambiguous-request
02	The user asked for an email. Is that the requirement?	Bridge	week-02.html#/elicitation-and-scope
03	Turn “short booking” into a rule you can examine	Anchor	week-02.html#/observable-behavior
04	“Fast” needs a workload and a measurement boundary	Bridge	week-02.html#/quality-with-conditions
05	A precise requirement can still describe the wrong thing	Bridge	week-02.html#/acceptance-and-validation
06	A requirement should lead somewhere	Anchor	week-02.html#/traceability-matrix
07	The loudest request is not automatically the next one	Bridge	week-02.html#/priority-and-risk
08	When the rule changes, what happens to its evidence?	Bridge	week-02.html#/requirements-under-change

Campus Rooms is a fictional room-booking service; every name, number, request and date is a teaching example unless a source is named. Rules and values marked example assumption belong to that example. Temporary variants are comparisons only: the canonical rules stay as the register states them.

One sentence arrived. What would you build from it?

Week 1 ended with a request from the people who run the rooms. Before any code, decide what it asks for.

The situation

The room staff sent one sentence at the end of Week 1: "Make room booking fair and easy." No examples, no numbers, no names. A first reading, written confidently: fair means whoever asks first gets the room; easy means fewer clicks. Nothing in it is wrong yet, and nothing in it is decided.

The question of this week

How would we know that a requirement has been satisfied? (A) When the person who asked says it is done. (B) When a concrete example behaves the way the requirement says. (C) When the code runs without errors.

What this lecture builds

A small connected set of requirements for Campus Rooms: R-01 no overlapping confirmations, R-02 duration 30 to 120 minutes, R-03 who may cancel, R-04 notification is separate, R-05 keyboard completion and Q-01 search response time. Each has an example you can check, and each links to a design decision and an independent check.

Where this applies

Every request you ever receive in words has this shape: a wish, a few missing decisions and an assumption or two. The care you take with it grows with how many people will live with the result.

The thread

A precise requirement is also what makes an automated assistant's draft judgeable: scene 3 hands the same request to an assistant twice, once vague and once as R-02, and checks both drafts against one example.

What you already know, and where this lecture goes

How would we know that a requirement has been satisfied?

Start from a request you received in words

Think of the last time someone asked you for something in one sentence: make it faster, make it simple, make it fair. What did you build first? Which words did you fill in without asking? How did you know whether they got what they needed?

Four kinds of statement

Four kinds of statement will come apart this week: what the service must do (a behavior), how well it must do it (a quality target), a rule the organization sets (a business rule), and a way of building it (an implementation decision). Only the first three are requirements.

The thread

A precise requirement is also what makes an automated assistant's draft judgeable: scene 3 hands the same request to an assistant twice, once vague and once as R-02, and checks both drafts against one example.

Where the register goes next

The requirements written this week return in Week 3 as models, in Week 8 as checks and in Week 12 when they change.

By the end of this lecture you can

- 1 Identify ambiguity, missing stakeholders, and hidden assumptions in a request.
- 2 Turn a request into observable behavior with boundary and failure cases.
- 3 Distinguish a requirement from a solution decision and a supporting assumption.
- 4 Connect a requirement to a design decision and an independent verification check.
- 5 Explain how a changed requirement affects linked evidence.

One sentence, three systems

Panel 1 of 2 · One confident reading, built, and the two objections it met

Condition: Baseline: the developer reads the sentence alone and builds it State: 6 of 7

Three rules, three different winners, one sentence, and all three could be defended from the same six words. The questions that were never asked: **Who decides? What is scarce? What outcome matters?**

THE REQUEST

"Make room booking fair and easy."

Three requests for C101, 10:00-11:00: Ari at 09:00 · Bo at 09:05 · Room staff at 09:10

BUILT FROM THE REWRITE

The screen confirms the earliest request and turns the others away.

First come, first served

Ari · 09:00 · 4 this week

✓ Confirmed

Bo · 09:05 · 0 this week

✗ not available

Room staff · 09:10

✗ not available

Works exactly as written. Nobody else was asked.

THE DEVELOPER'S REWRITE

MORE CONFIDENT WORDING

"Rooms are allocated strictly in the order requests arrive."

DEPARTMENT COORDINATOR · OBJECTS AFTER THE BUILD

"Nobody should hold more than a fair share of the week."

Per-user weekly limit

Ari · 09:00 · 4 this week

✗ not available

Bo · 09:05 · 0 this week

✓ Confirmed

Room staff · 09:10

✗ not available

Winner: Bo, Ari is at the weekly limit.

ROOM STAFF · OBJECT AFTER THE BUILD

"Our requests have to come first; we run the rooms."

Staff requests take priority over student bookings

Ari · 09:00 · 4 this week

✗ not available

Bo · 09:05 · 0 this week

✗ not available

Room staff · 09:10

✓ Confirmed

Winner: Room staff, a staff request beats every student request.

NEVER ASKED:

Who decides?

What is scarce?

What outcome matters?

Three winners from one sentence: Ari, Bo, Room staff.

The rewrite "Rooms are allocated strictly in the order requests arrive." chose one reading of fair. Three requests for C101 at 10:00-11:00: under first come, first served Ari wins; under a weekly limit of 4 Bo wins; under staff priority the staff request wins. The questions never asked: Who decides? What is scarce? What outcome matters?

Week 02 · week-02.html · #/ambiguous-request

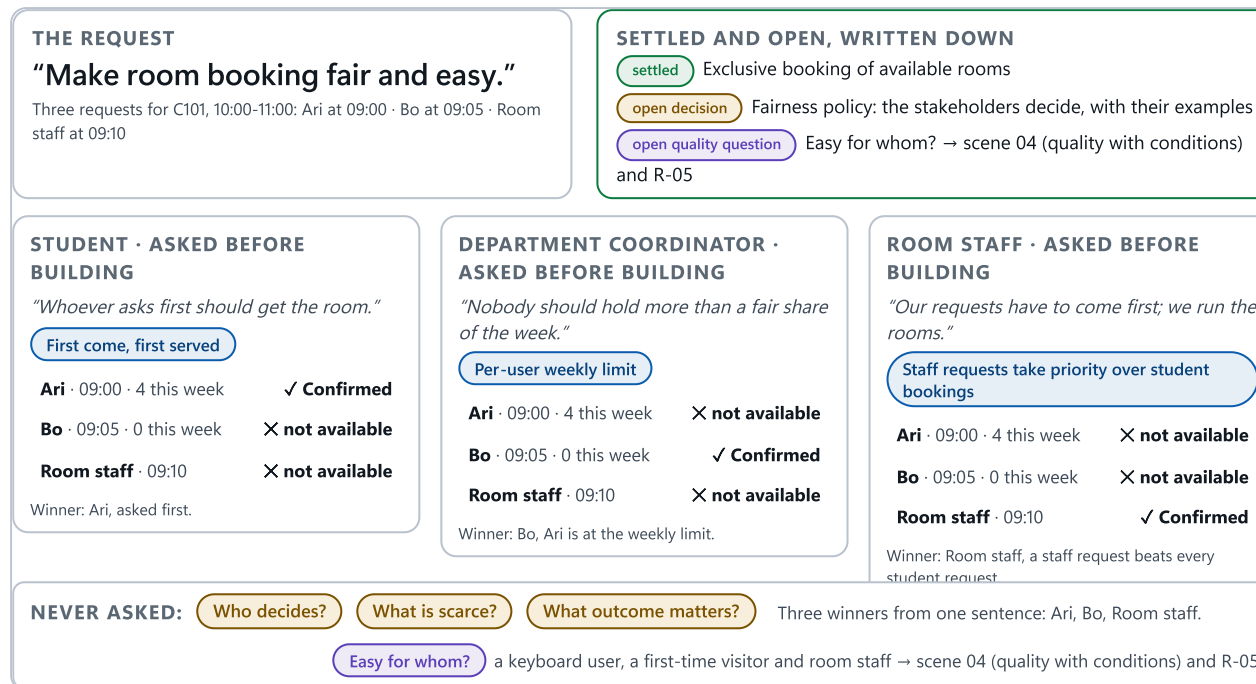
Anchor · page 4

One sentence, three systems

Panel 2 of 2 · The same sentence read with each stakeholder first: settled scope and open decisions

Condition: Changed condition: each stakeholder is asked before anything is built State: 6 of 7

Settled for this lecture: **exclusive booking of available rooms**. Left open, on purpose: the fairness policy, recorded as a decision the stakeholders still owe. And **easy for whom?** goes to scene 4 and Rule R-05 (EXAMPLE ASSUMPTION).



Settled for this lecture: exclusive booking of available rooms; fairness policy remains an explicit open decision. Easy is a separate ambiguity (easy for whom? a keyboard user, a first-time visitor and room staff), routed to scene 04 (quality with conditions) and R-05. The requests, counts and limit are simulated example values.

Principle · Ambiguity is resolved through examples and stakeholder decisions, not through more confident wording.

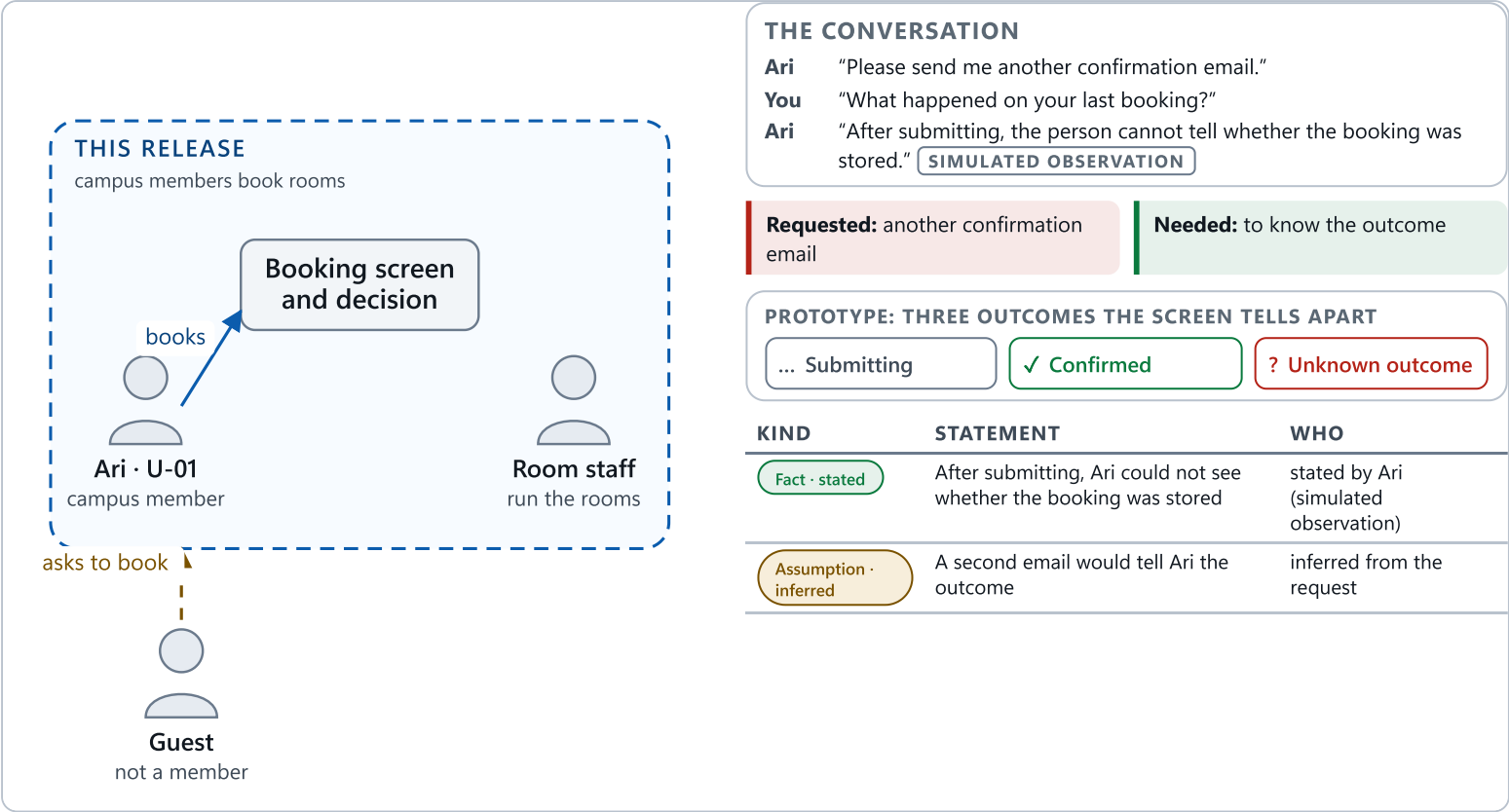
In short · Confident wording settles nothing; examples and decisions do.

The user asked for an email. Is that the requirement?

Panel 1 of 3 · From a requested email to the need behind it

Condition: Baseline: a campus member asks for a second confirmation email State: 5 of 8

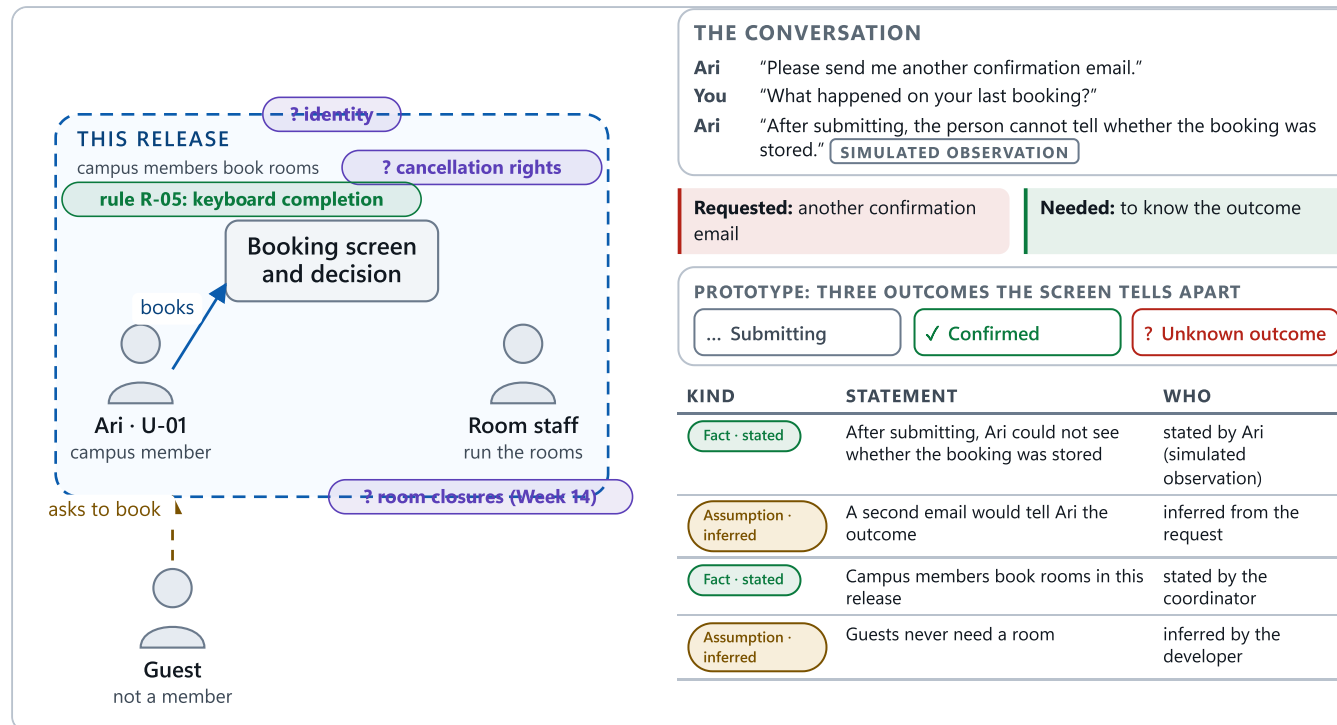
A small prototype separates three outcomes on the screen itself: **Submitting**, **Confirmed**, **Unknown outcome**. A prototype walkthrough informs the requirement; the authoritative stored result is a separate concern.



Ari asked for another confirmation email; the question “What happened on your last booking?” produced a simulated observation: after submitting, the person cannot tell whether the booking was stored. The need is to know the outcome. The prototype tells Submitting, Confirmed, Unknown outcome apart; a prototype walkthrough informs the requirement; the authoritative stored result is a separate concern.

Panel 2 of 3 · Questions at the boundary and the register of facts, assumptions and open questions

Facts have a speaker, assumptions have an author, questions have a place on the line. The register makes each of them something the team can talk about.



Before accepting a guest's request: Who is this person to the campus, and how does the service know? May a guest cancel, and whose booking? Can the guest complete the booking by keyboard, with status they can perceive? Who may record a room closure, and what happens to affected bookings? Facts carry a speaker, assumptions an author, questions a place on the line.

Principle · An explicit boundary and an open-question register make uncertainty discussable.

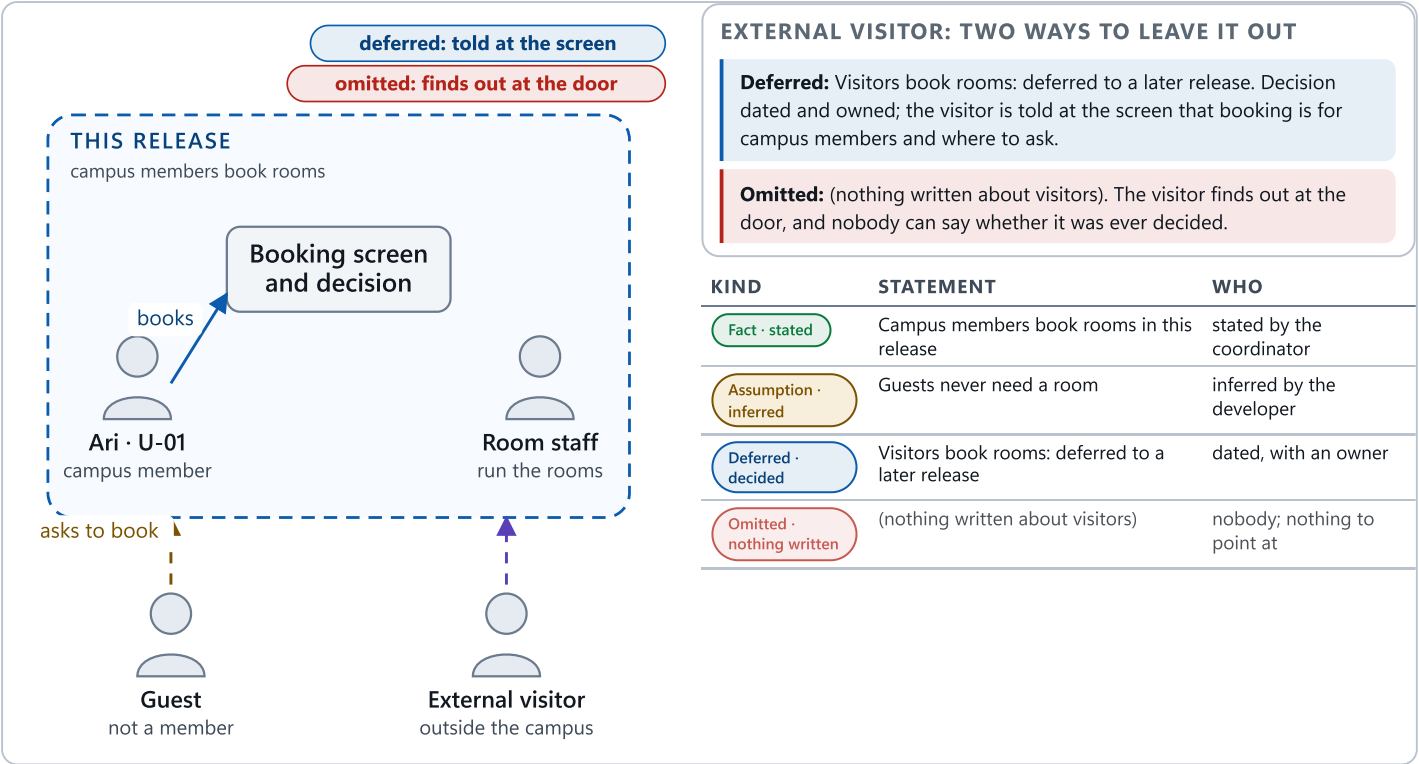
In short • Write down what you know, what you assume, and what is open.

The user asked for an email. Is that the requirement?

Panel 3 of 3 · A visitor appears: deferred explicitly, or omitted by accident

Condition: Changed condition: an external visitor appears; the boundary stays State: 4 of 4

Same boundary, same visitor. A deferred behavior is a decision with a date and an owner; an omitted one is a surprise waiting at the door.



Deferred: Visitors book rooms: deferred to a later release, decision dated and owned; the visitor is told at the screen that booking is for campus members and where to ask. Omitted: (nothing written about visitors), the visitor finds out at the door, and nobody can say whether it was ever decided.

Principle · An explicit boundary and an open-question register make uncertainty discussable.

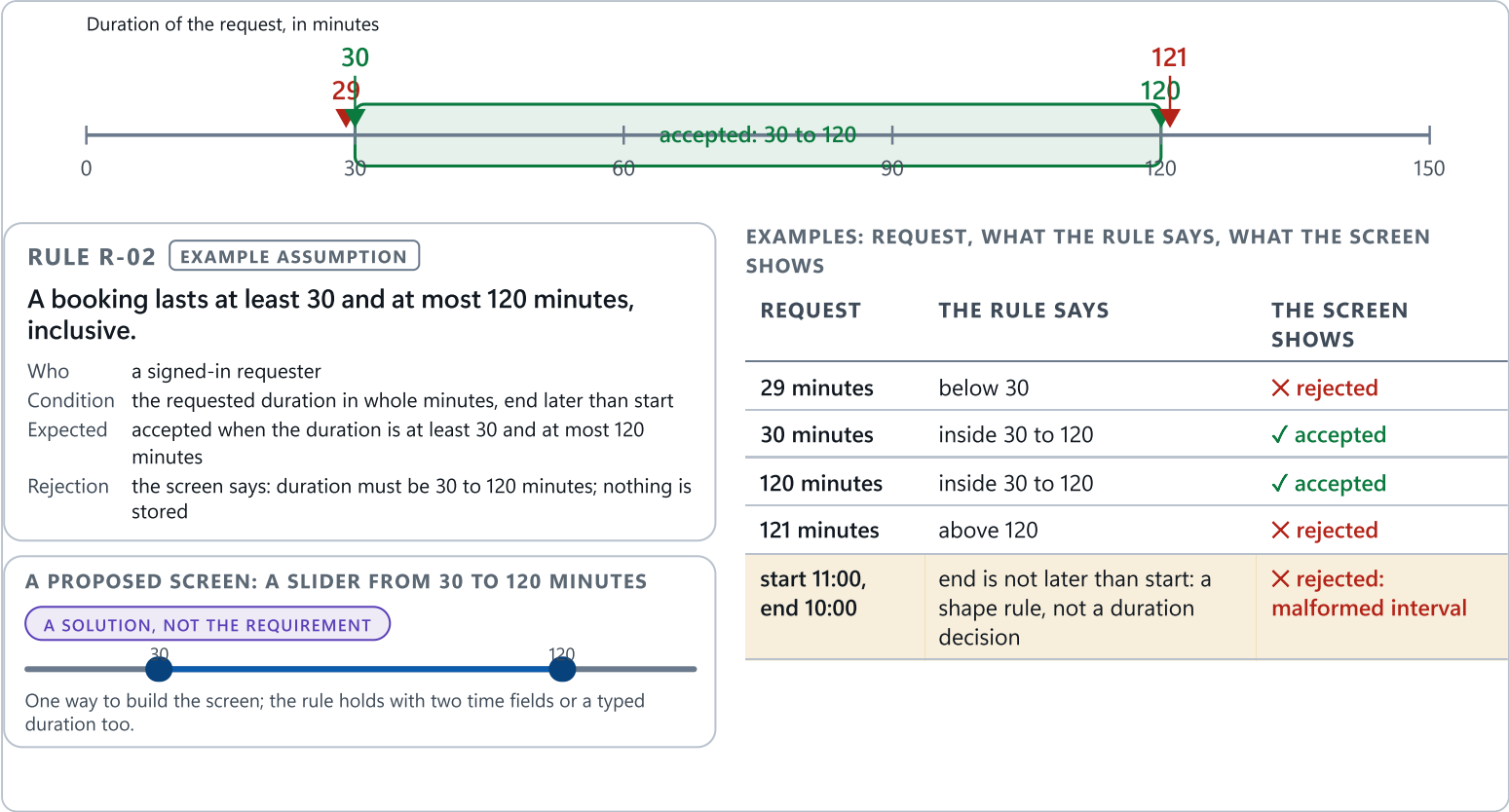
In short · Write down what you know, what you assume, and what is open.

Turn “short booking” into a rule you can examine

Panel 1 of 3 · The rule, its four parts and the examples at both ends

Condition: Baseline: rule R-02, 30 through 120 minutes inclusive State: 6 of 7

One more request: start 11:00, end 10:00. It is **rejected as a malformed interval, not a duration decision**. One boundary rule does not describe every validity condition; the register needs the shape rule too.



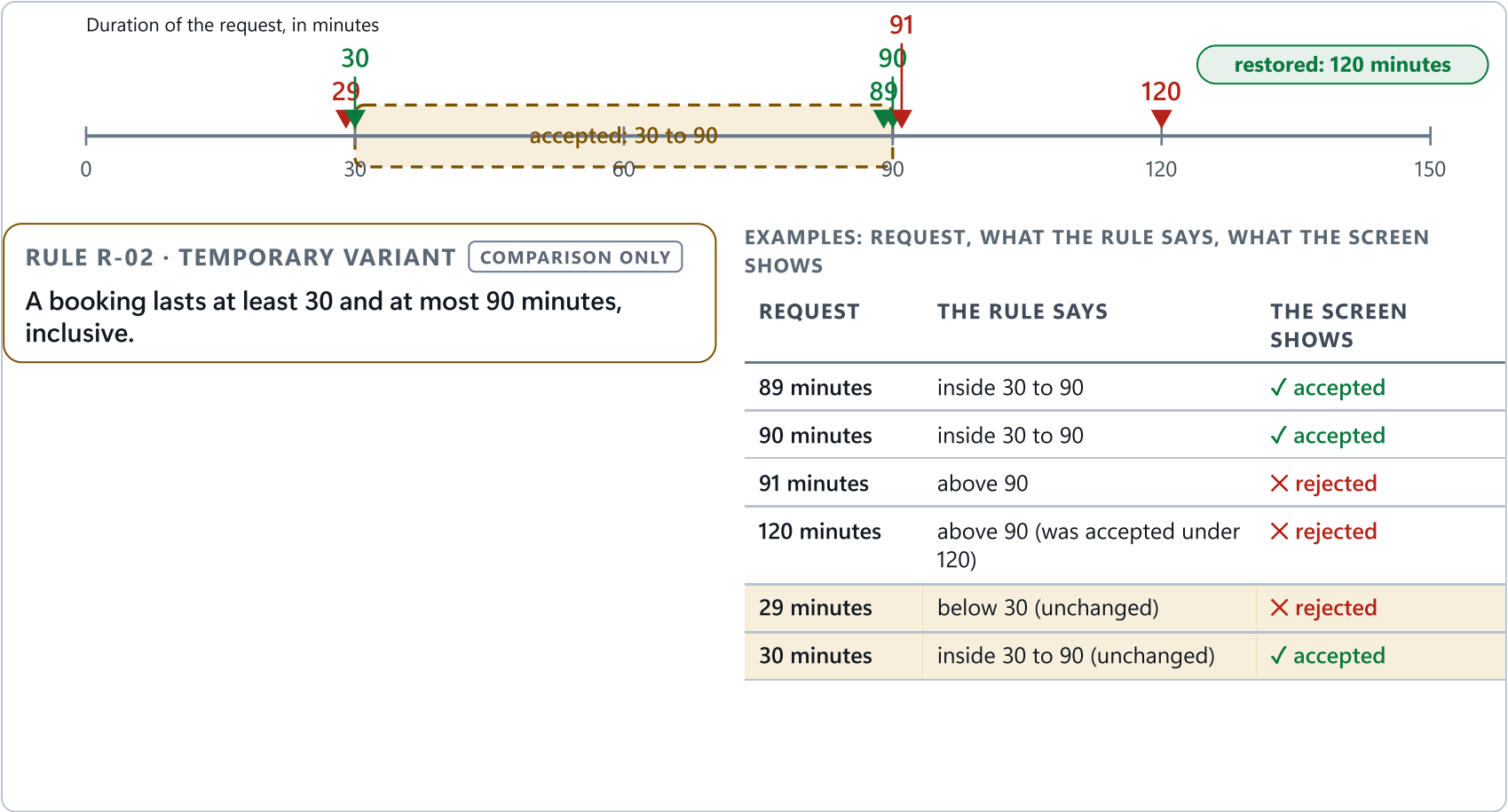
R-02 in four parts: who (a signed-in requester); condition (the requested duration in whole minutes, end later than start); expected (accepted when the duration is at least 30 and at most 120 minutes); rejection (the screen says: duration must be 30 to 120 minutes; nothing is stored). The examples: 29 rejected, 30 accepted, 120 accepted, 121 rejected; start 11:00 end 10:00 rejected as a malformed interval, not a duration decision. The slider is a solution, not the requirement.

Turn “short booking” into a rule you can examine

Panel 2 of 3 · The temporary 90-minute variant, a comparison only

Condition: Temporary variant: upper limit 90 minutes, a comparison only State: 3 of 4

The rows at the lower end, 29 and 30, are unchanged: the lower limit was not touched. Then the canonical rule is restored: **120 minutes**.

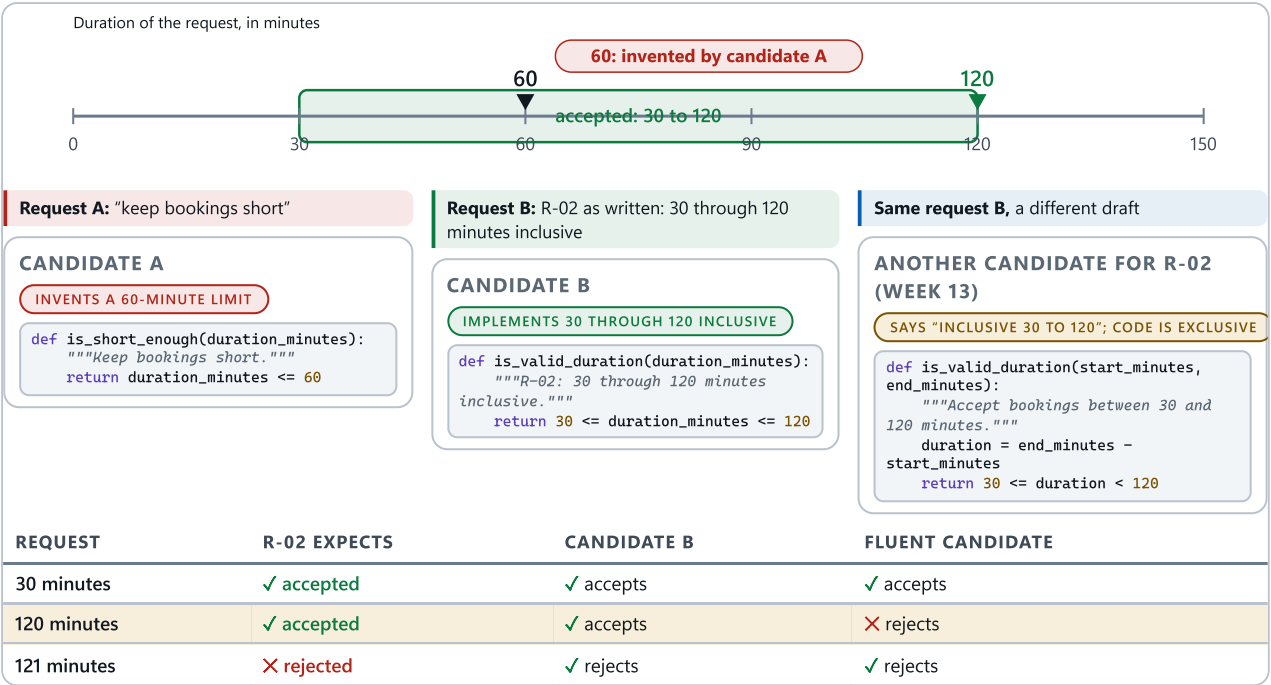


Only the upper limit changes: 89 accepted, 90 accepted, 91 rejected; 120 flips to rejected; 29 and 30 are unchanged. Afterwards the canonical rule is restored: 120 minutes.

Panel 3 of 3 · Two requests to an assistant, three candidates, one deciding example

Condition: Changed condition: two drafting requests to an assistant, one vague and one with R-02 State: 6 of 6

A precise requirement makes a draft checkable; the check still has to be made. The rule made an independent judgment possible, and the 120-minute example made it.



Request A (“keep bookings short”) produced a candidate that invents a 60-minute limit; request B (R-02 as written) produced one that implements 30 through 120 inclusive; a third candidate for the same request explains “inclusive 30 to 120” but codes an exclusive bound. The 120-minute example settles it. The candidates are authored fixtures, not the output of a live assistant.

Principle · A useful requirement makes meaningful differences between acceptable and unacceptable outcomes visible.

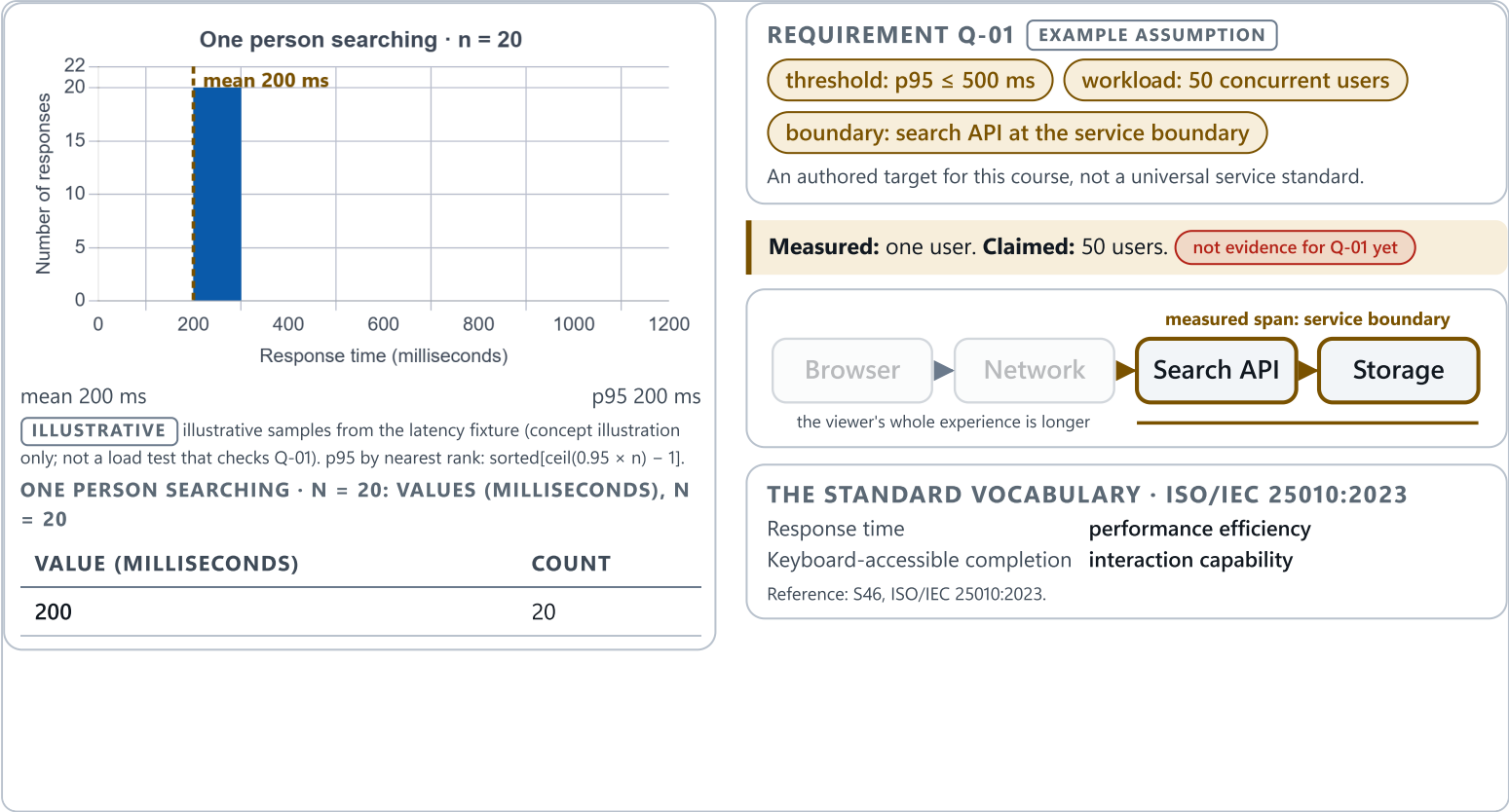
In short · A good rule shows exactly where accepted turns into rejected.

“Fast” needs a workload and a measurement boundary

Panel 1 of 3 · One number, then the requirement with its threshold, workload and boundary

Condition: Baseline: one person searching; twenty responses sampled (illustrative samples) State: 6 of 7

The names other engineers use: ISO/IEC 25010:2023 calls response time **performance efficiency** and keyboard-accessible completion **interaction capability**. Same ideas, shared words.



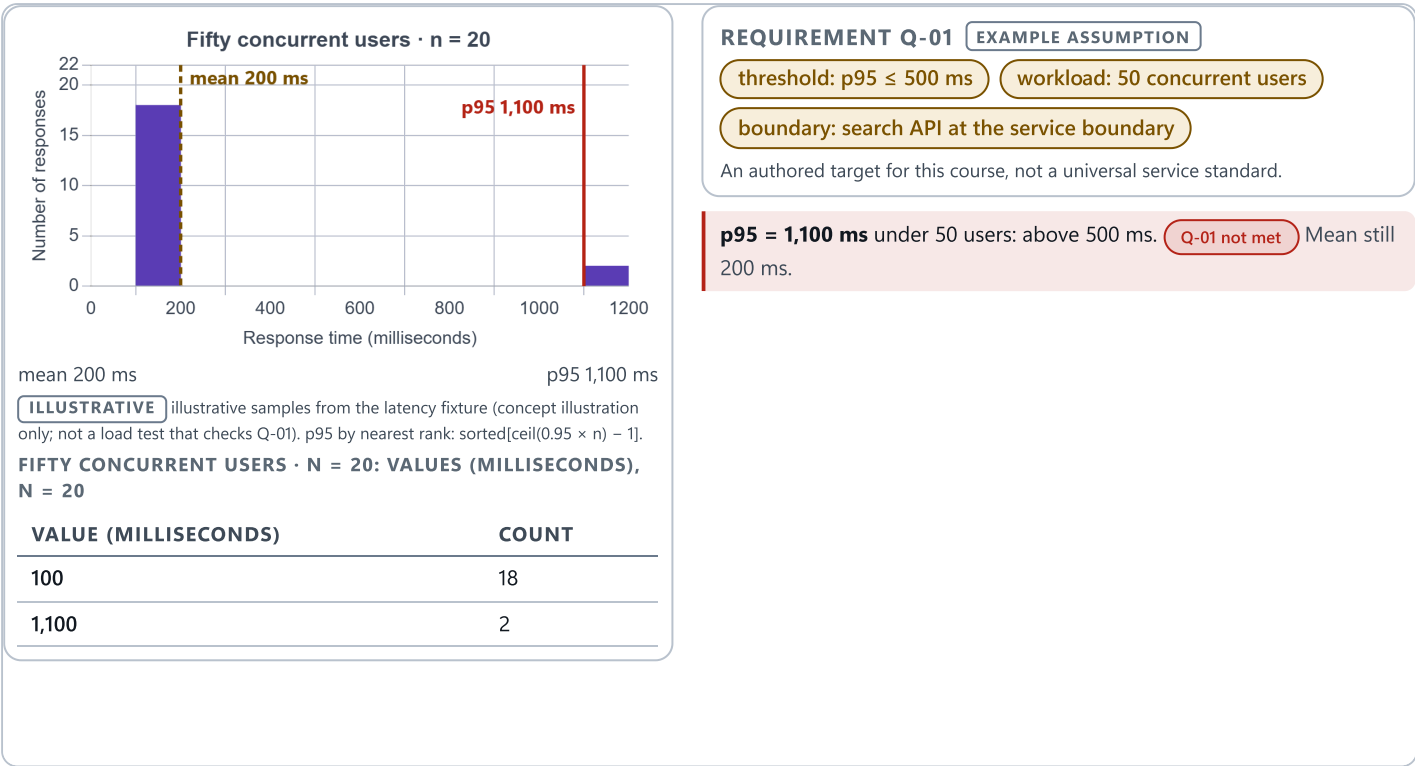
Q-01: p95 ≤ 500 ms under 50 concurrent users at the search API at the service boundary; an authored target. The sample of 20 responses under one user (all 200 ms) is illustrative and says nothing about Q-01. Vocabulary: Response time is performance efficiency; Keyboard-accessible completion is interaction capability (ISO/IEC 25010:2023).

"Fast" needs a workload and a measurement boundary

Panel 2 of 3 · The same target under fifty users: the mean hides the tail, the percentile shows it

Condition: Changed condition: fifty concurrent users; twenty responses sampled; same target, same code (illustrative samples) State: 3 of 3

The average hid the two slow responses; the percentile with its conditions did not. Stating the workload is what made the difference visible.



18 responses at 100 ms and 2 at 1,100 ms: mean 200 ms, nearest-rank p95 1,100 ms, above the 500 ms threshold. Illustrative samples from the latency fixture, not a load test.

Principle · A quality requirement needs observable acceptance, a boundary, and stated conditions.

In short · Fast means nothing without a workload, a boundary and a threshold.

"Fast" needs a workload and a measurement boundary

Panel 3 of 3 · A categorical quality: the keyboard path of R-05

Condition: Changed condition: a different quality, rule R-05: complete the booking with the keyboard alone State: 6 of 6

Acceptance here is categorical: **reachable and understandable**, yes or no. Q-01 accepts by a number; R-05 accepts by an outcome. Both are observable.

campus-rooms.example/book

Ari · keyboard only

ROOM LIST

C101

TIME FIELDS

10:00–11:00

Reserve

Confirmed: C101, 10:00–11:00

Status message is announced (live region).

Keys pressed:

Tab

Tab

Tab

Enter

REQUIREMENT Q-01

EXAMPLE ASSUMPTION

threshold: $p95 \leq 500\text{ ms}$

workload: 50 concurrent users

boundary: search API at the service boundary

An authored target for this course, not a universal service standard.

THE KEYBOARD PATH, R-05

Tab

 → the room list

Tab

 → the time fields

Tab

 → the Reserve button

Enter

 → Confirmed: the status message is announced

TWO REQUIREMENTS, TWO KINDS OF ACCEPTANCE

REQUIREMENT	ACCEPTED WHEN	KIND
Q-01 search response	$p95 \leq 500\text{ ms}$ under 50 users at the service boundary	numerical
R-05 keyboard completion	the outcome is reachable and understandable	categorical

Tab: the room list; Tab: the time fields; Tab: the Reserve button; Enter: Confirmed: the status message is announced. Accepted when the outcome is reachable and understandable, yes or no.

Principle · A quality requirement needs observable acceptance, a boundary, and stated conditions.

In short · Fast means nothing without a workload, a boundary and a threshold.

A precise requirement can still describe the wrong thing

Panel 1 of 2 · The system matches its statement; the statement does not match the need

Condition: Baseline: the system is checked against its written statement State: 5 of 7

Before accepting a specification, compare the proposed behavior with one concrete stakeholder example. Here that example is 120 minutes, and it would have failed on the first read.

THE NEED
Ari (U-01): a two-hour study session in C101 from 10:00 to 12:00.
120 minutes must be accepted

mismatch with the statement

×

THE WRITTEN STATEMENT
Booking duration is at most 30 minutes.
the intended 120 was written down as 30

→

THE OBSERVED BEHAVIOR
Every request longer than 30 minutes is rejected; the two-hour session cannot be booked.

CHECKS: WHAT THE WORDS SAY AGAINST WHAT THE SYSTEM DOES

REQUEST	THE WORDS SAY	THE SYSTEM DOES	RESULT
30 minutes	statement: accepted	system: accepted	✓ match
31 minutes	statement: rejected	system: rejected	✓ match
120 minutes	statement: rejected	system: rejected	✓ match
120 minutes	the need: accepted	system: rejected	✗ mismatch

Feature delivered, outcome missed: every reservation is short, and the two-hour session is impossible.

Before accepting a specification: compare the proposed behavior with one concrete stakeholder example. Here: 120 minutes.

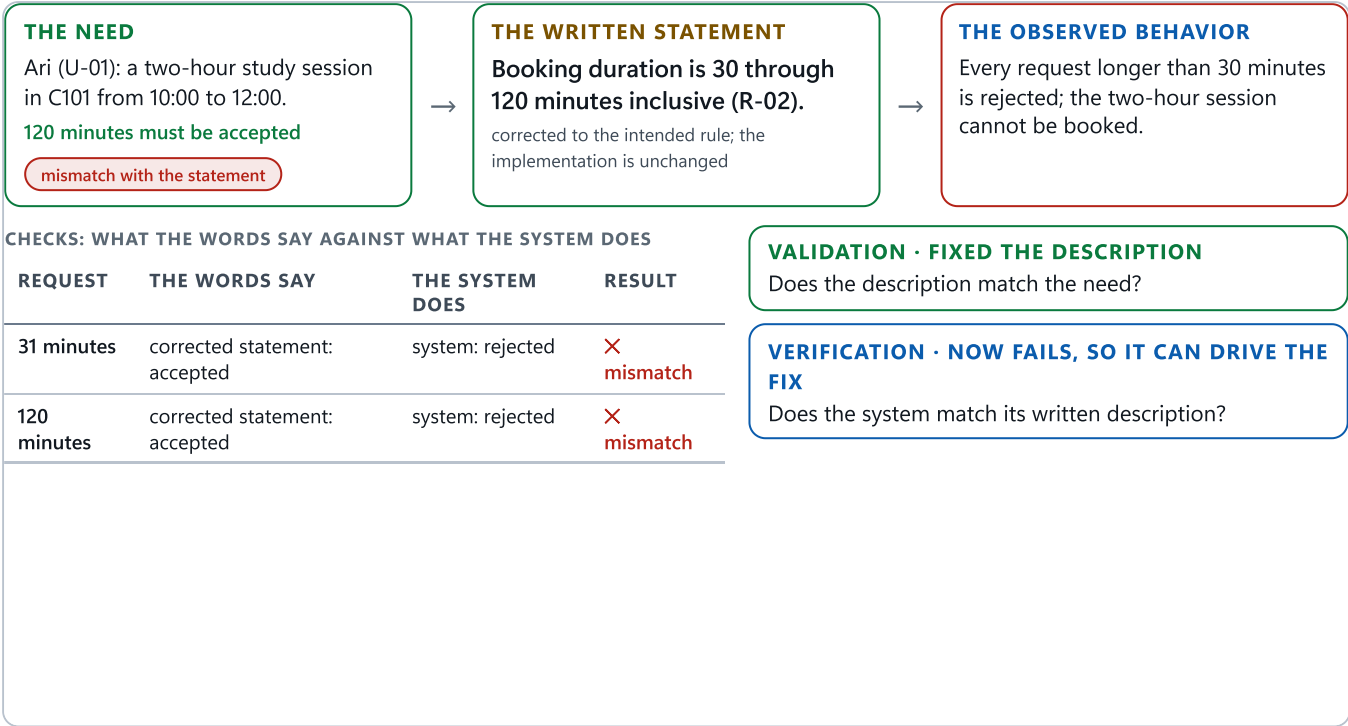
Verification: 30 minutes accepted as the statement says; 31 minutes rejected as the statement says; 120 minutes rejected as the statement says. Validation: Ari's a two-hour study session in C101 from 10:00 to 12:00 needs 120 minutes and is rejected. The intended 120 was written down as 30.

A precise requirement can still describe the wrong thing

Panel 2 of 2 · Two questions, and the corrected statement that makes verification fail visibly

Condition: Changed condition: the implementation stays; the intended requirement is corrected State: 3 of 3

Correcting the words moved the mismatch to where checks can see it. Validation fixed the description; verification will now drive the implementation.



A requirement should lead somewhere

Panel 1 of 2 · The matrix with three rows and the chain of each

Condition: Baseline: the matrix grows one requirement at a time State: 7 of 8

Why a matrix and not a pipeline diagram: coverage is a question about rows and blanks, and a blank cell is visible here. Its limit: **a requirement nobody wrote has no row: a matrix shows the links it has, not the ones it lacks.**

TRACEABILITY MATRIX: REQUIREMENT · DESIGN DECISION · EXAMPLES · CHECKS

REQUIREMENT	DESIGN DECISION	EXAMPLES	CHECKS
R-01 from Week 1	authoritative conflict decision at the booking boundary	- C101 [10:00,11:00) vs [10:30,11:30) conflicts [11:00,12:00) is adjacent and allowed - C202 at the same time is allowed	- interval predicate check - concurrent-request check at the storage boundary
R-03 cancellation authorization	ownership read from the stored booking, not from the request	- U-01 cancels own future booking: allowed - U-02 cancels B-001: denied - U-01 at 10:00: denied	authorization matrix check
R-02 from scene 03	whole-minute duration validated once, at the request boundary	29 minutes: rejected 30 and 120 minutes: accepted 121 minutes: rejected	duration boundary check
R-??	— no row —	a requirement nobody wrote has no row: a matrix shows the links it has, not the ones it lacks	—

THE CHAIN OF THE SELECTED ROW

Click a requirement or a cell to light up its chain.

needs review after the change

unchanged

no row: nothing to trace

WHY A MATRIX

Coverage is a question about rows and blanks. A pipeline diagram shows movement; a matrix shows what is missing.

A requirement nobody wrote has no row: a matrix shows the links it has, not the ones it lacks.

R-01: authoritative conflict decision at the booking boundary; examples C101 [10:00,11:00) vs [10:30,11:30) conflicts; [11:00,12:00) is adjacent and allowed; C202 at the same time is allowed; checks interval predicate check, concurrent-request check at the storage boundary. R-03: ownership read from the stored booking, not from the request; examples U-01 cancels own future booking: allowed; U-02 cancels B-001: denied; U-01 at 10:00: denied; checks authorization matrix check. R-02: whole-minute duration validated once, at the request boundary; examples 29 minutes: rejected; 30 and 120 minutes: accepted; 121 minutes: rejected; checks duration boundary check. A requirement nobody wrote has no row: a matrix shows the links it has, not the ones it lacks.

A requirement should lead somewhere

Panel 2 of 2 · The variant highlights only what depends on R-03

Condition: Variant: delegated staff cancellation with a recorded reason (requires a stakeholder decision) State: 4 of 4

The variant is marked as **stakeholder decision**: the matrix shows what a decision would cost, and the decision belongs to the stakeholders.

TRACEABILITY MATRIX: REQUIREMENT · DESIGN DECISION · EXAMPLES · CHECKS

REQUIREMENT	DESIGN DECISION	EXAMPLES	CHECKS
R-01 from Week 1	authoritative conflict decision at the booking boundary	- C101 [10:00,11:00) vs [10:30,11:30) conflicts [11:00,12:00) is adjacent and allowed - C202 at the same time is allowed	- interval predicate check - concurrent-request check at the storage boundary ✓ unchanged: interval checks
R-03 cancellation authorization	ownership read from the stored booking, not from the request ⌚ needs review: R-03 decision	- U-01 cancels own future booking: allowed - U-02 cancels B-001: denied - U-01 at 10:00: denied ⌚ needs review: cancellation decision table	authorization matrix check ⌚ needs review: authorization matrix check
R-02 from scene 03	whole-minute duration validated once, at the request boundary	29 minutes: rejected 30 and 120 minutes: accepted 121 minutes: rejected	duration boundary check ✓ unchanged: duration checks

THE CHAIN OF THE SELECTED ROW

Click a requirement or a cell to light up its chain.

needs review after the change

unchanged

no row: nothing to trace

PROPOSED VARIANT OF R-03

REQUIRES A STAKEHOLDER DECISION

Delegated staff cancellation with a recorded reason.

⌚ R-03 decision, authorization matrix check, cancellation decision table

✓ duration checks, interval checks

Delegated staff cancellation with a recorded reason: needs review R-03 decision, authorization matrix check, cancellation decision table (marked ⌚); unchanged duration checks, interval checks (marked ✓). The variant requires a stakeholder decision.

Principle · Traceability makes relationships inspectable; a link is useful only when its meaning and evidence are current.

In short · A link is only worth what its evidence is today.

The loudest request is not automatically the next one

Panel 1 of 2 · The decision table and the order it explains

Condition: Baseline: an ordinary week, no closure planned State: 6 of 7

The order: **P1** → **P3** → **P2**. P1 removes a broken promise and keeps R-01; P3 is small and removes an exclusion; P2 changes no outcome and waits for the photo owner. No number produced this; the reasons did, and they are on the table.

DECISION TABLE (QUALITATIVE; NO SCORE)					
REQUEST	USER BENEFIT	DEPENDENCY	UNCERTAINTY	CONSEQUENCE OF WAITING	EFFORT
P1 Correct duplicate confirmations	one confirmed booking per room and hour (R-01); nobody meets a stranger at the door	the shared booking decision from Week 1	low: the cause is known	two people at one door for every overlap that slips through	Medium
P2 Add decorative room photos	a nicer screen; no task completes sooner	room records must supply the photos	medium: who owns the photos is not settled	none for any booking outcome	Small to medium
P3 Make cancellation work by keyboard (R-05)	keyboard users can cancel at all	the cancellation screen (R-03)	low: the route is known from the keyboard walk-through	a room stays booked by someone who cannot cancel it, and nobody else can book it	Small
ORDER: 1. Correct duplicate confirmations → 2. Make cancellation work by keyboard (R-05) → 3. Add decorative room photos P1 removes a broken promise and keeps R-01; P3 is small and removes an exclusion; P2 changes no outcome and waits for the photo owner.					

Columns: User benefit, Dependency, Uncertainty, Consequence of waiting, Effort. Order P1 → P3 → P2: P1 removes a broken promise and keeps R-01; P3 is small and removes an exclusion; P2 changes no outcome and waits for the photo owner.

Panel 2 of 2 · A closure tomorrow changes the urgency, not the value

New order: **P3** → **P1** → **P2**. The closure adds urgency to cancellation; the benefit, dependency, uncertainty and effort cells did not change.

A room closure is scheduled for tomorrow (fictional scenario; recording closures stays an open need until Week 14). Urgency: P1 unchanged; P2 unchanged; P3 rises: people must cancel and rebook before tomorrow, keyboard users included. Order P3 → P1 → P2: The closure adds urgency to cancellation; the benefit, dependency, uncertainty and effort cells did not change.

Principle · Priority is a reasoned choice under constraints, not a permanent property of a feature.

In short · Order is a decision with reasons, not a label on a feature.

When the rule changes, what happens to its evidence?

Panel 1 of 2 · The rule changes: new wording, rationale, dated decision, and the evidence that goes stale

Condition: Baseline: rule change: the upper limit becomes 90 minutes (a scenario; the canonical R-02 stays 30 to 120) State: 5 of 6

What travels together: the new wording, its rationale, the dated decision, its owner, the two stale items marked for revision, and the old wording kept as history. Nothing pretends to be current.

R-02 · V1

HISTORY

"Booking duration is 30 through 120 minutes inclusive."

Decided 8 September 2026 (fictional) · owner: room policy owner (fictional role)

R-02 · V2

RULE CHANGE

SCENARIO

"Booking duration is 30 through 90 minutes inclusive."

Decided 14 October 2026 (fictional) · owner: room policy owner (fictional role)

Rationale: more people need rooms in the afternoon peak; shorter sessions let more of them fit (fictional rationale)

V1 stays readable as history, with its date.

LINKED EVIDENCE AND ITS STATUS

E1	EXAMPLE 29 minutes: rejected	current
E2	EXAMPLE 30 minutes: accepted	current
E3	EXAMPLE 120 minutes: accepted	stale · 120 minutes must now be rejected
E4	EXAMPLE 121 minutes: rejected	current
C1	CHECK duration boundary check: 30 and 120 inclusive	stale · the upper bound in the check must change
C2	CHECK interval overlap check (R-01)	current

WHAT TRAVELS TOGETHER

- ✓ the new wording, in place
- ✓ its rationale, written next to it
- ✓ a dated decision with an owner
- ✓ E3 revised: 120 minutes must now be rejected
- ✓ C1 rewritten: the upper bound in the check must change
- ✓ the old wording kept as history

Rule change: the upper limit becomes 90 minutes: "Booking duration is 30 through 90 minutes inclusive." Rationale: more people need rooms in the afternoon peak; shorter sessions let more of them fit (fictional rationale). Evidence: E1 current; E2 current; E3 stale: 120 minutes must now be rejected; E4 current; C1 stale: the upper bound in the check must change; C2 current. V1 stays readable as history, with its date. This is a change-management scenario with fictional dates; the course's canonical R-02 stays 30 through 120 minutes.

When the rule changes, what happens to its evidence?

Panel 2 of 2 · A wording correction: the same behavior, every item stays current

Condition: Changed condition: wording correction: the same behavior in a clearer sentence State: 3 of 3

A semantic change and an editorial change look alike in a diff and mean different things. Judge the meaning, then decide what to revise.

R-02 · V1

CURRENT

"Booking duration is 30 through 120 minutes inclusive."
Decided 8 September 2026 (fictional) · owner: room policy owner (fictional role)

R-02 · V1A

WORDING CORRECTION

"A booking lasts at least 30 and at most 120 minutes, both inclusive."
Decided 14 October 2026 (fictional) · owner: room policy owner (fictional role)
Rationale: the words 'through 120 inclusive' were misread twice in review (fictional rationale)

The old wording is kept in the history note; no example or check changes.

LINKED EVIDENCE AND ITS STATUS

E1	EXAMPLE 29 minutes: rejected	current
E2	EXAMPLE 30 minutes: accepted	current
E3	EXAMPLE 120 minutes: accepted	current
E4	EXAMPLE 121 minutes: rejected	current
C1	CHECK duration boundary check: 30 and 120 inclusive	current
C2	CHECK interval overlap check (R-01)	current

WHAT TRAVELS TOGETHER

- ✓ the clearer wording, in place
- ✓ its rationale, written next to it
- ✓ a dated decision with an owner
- examples: unchanged, every outcome the same
- checks: unchanged, nothing to rewrite
- ✓ the old wording kept in the history note

"A booking lasts at least 30 and at most 120 minutes, both inclusive." Rationale: the words 'through 120 inclusive' were misread twice in review (fictional rationale). All six items stay current; the old wording is kept in the history note; no example or check changes.

Principle · Manage the meaning of a change, its rationale, and its evidence together.

In short · Change the rule, its reason and its evidence in one move.

The requirement register of this lecture

Six statements with identifiers that reappear in later weeks, four open items kept visible, and three sentences that are not requirements at all.

ID	KIND	STATEMENT	ACCEPTANCE EXAMPLE	STATUS
R-01	behavior	No overlapping confirmed bookings for the same room; intervals are half-open, so adjacent bookings do not overlap.	C101 [10:00,11:00) vs [10:30,11:30) conflicts	stated, example-backed, traced
R-02	behavior	Booking duration is 30 through 120 minutes inclusive, whole minutes, end later than start.	29 rejected, 30 accepted, 31 accepted, 119 accepted, 120 accepted, 121 rejected	stated, example-backed, traced
R-03	business rule	An authenticated requester may cancel their own future confirmed booking; started and other-owner cases are denied.	U-01 cancels own B-001 at 09:00 for 10:00: allowed; U-02: denied	stated, example-backed, traced; delegated cancellation is an open variant
R-04	behavior	Notification failure does not undo a confirmed booking; pending or failed messages stay visible and recoverable.	provider unavailable: booking stays confirmed, screen says the message is delayed (Week 1)	stated, example-backed
R-05	quality (accessibility)	The core task can be completed with the keyboard, with status and error information the person can perceive.	Tab, Tab, Tab, Enter: booking completed, status announced	stated, example-backed; acceptance is categorical
Q-01	quality target	p95 of search response time at most 500 ms under 50 concurrent users, at the search API at the service boundary (teaching fixture).	20 samples under fifty users: p95 1,100 ms, not met (illustrative)	stated; threshold, workload, boundary named; not yet checked under its conditions
·	open decision	The fairness policy: first come first served, a per-user limit or staff priority.	scene 1	open; the stakeholders decide
·	open quality question	Easy for whom: a keyboard user, a first-time visitor and room staff.	scene 1 → scene 4, R-05	open
·	deferred	Visitors book rooms: deferred to a later release.	scene 2	decided, dated, owned
·	open question	Who may record a room closure, and what happens to affected bookings.	scenes 2 and 7	open until Week 14

Not requirements

“Use Framework X to make booking fast.” A solution choice mixed with a vague quality objective; name the outcome and its conditions separately. (solution + vague quality)

“Use a relational database.” An implementation decision; it belongs in the design record next to the requirement it serves. (implementation decision)

“A slider from 30 to 120 minutes.” A screen design; the requirement behind it is R-02. (solution)

Eight principles, six checks

One principle per scene. Then six questions: decide your answer before you open it. The last one leaves room booking behind.

Principles

1 · ANCHOR

Ambiguity is resolved through examples and stakeholder decisions, not through more confident wording.

In short: Confident wording settles nothing; examples and decisions do.

week-02.html#/ambiguous-request

2 · BRIDGE

An explicit boundary and an open-question register make uncertainty discussable.

In short: Write down what you know, what you assume, and what is open.

week-02.html#/elicitation-and-scope

3 · ANCHOR

A usefult requirement makes meaningful differences between acceptable and unacceptable outcomes visible.

In short: A good rule shows exactly where accepted turns into rejected.

week-02.html#/observable-behavior

4 · BRIDGE

A quality requirement needs observable acceptance, a boundary, and stated conditions.

In short: Fast means nothing without a workload, a boundary and a threshold.

week-02.html#/quality-with-conditions

5 · BRIDGE

We must question both whether the system matches its description and whether the description matches the need.

In short: Check the system against the words, and the words against the need.

week-02.html#/acceptance-and-validation

6 · ANCHOR

Traceability makes relationships inspectable; a link is useful only when its meaning and evidence are current.

In short: A link is only worth what its evidence is today.

week-02.html#/traceability-matrix

7 · BRIDGE

Priority is a reasoned choice under constraints, not a permanent property of a feature.

In short: Order is a decision with reasons, not a label on a feature.

week-02.html#/priority-and-risk

8 · BRIDGE

Manage the meaning of a change, its rationale, and its evidence together.

In short: Change the rule, its reason and its evidence in one move.

week-02.html#/requirements-under-change

Checks: decide your answer before reading the one below each question

1. Is “Use Framework X to make booking fast” a clear behavior requirement?

It mixes a solution choice with an underspecified quality objective; define the needed outcome and conditions separately.

2. Should a 120-minute booking satisfy R-02?

Yes. The upper limit is inclusive; other validity and availability checks still apply.

3. What is missing from “Search must finish in 500 milliseconds”?

At least the measurement boundary, workload, proportion or percentile, and conditions.

4. What does selecting R-03 in the matrix tell us?

Which decisions and checks are linked to cancellation authorization; those links need review when the rule changes.

5. Does supplying R-02 to an assistant guarantee that its candidate is correct?

No. The explicit rule makes an independent judgment possible; the inclusive boundary still needs to be checked.

6. A desktop editor request says “save my work safely.” What must be clarified before choosing autosave?

Clarify what counts as saved, acceptable loss after interruption, recovery feedback and overwrite behavior. Choose an observable recovery example with the user; autosave is a possible design, not the requirement itself.

What carries forward, and where to read more

Next week turns the register into models: which picture answers which question about a booking.

Week 3 starts from this question:

Does a written rule alone make the sequence of booking, cancellation and failure states understandable?

Carried forward to Week 3

- The requirement register: R-01 to R-05 and Q-01, each with an acceptance example and a stable identifier.
- The traceability matrix: R-01, R-02 and R-03 linked to a design decision, examples and checks.
- The open items, kept visible: the fairness policy, easy for whom, visitors deferred, and room closures until Week 14.

In practice · examples from 2026

Requirements live as issues or user stories in GitHub Issues, Jira or Azure Boards, and acceptance examples are often written as Given/When/Then scenarios that tools such as Cucumber or pytest-bdd run as checks.

Sources for this lecture

- S01** IEEE Computer Society, Guide to the Software Engineering Body of Knowledge (SWEBOK V4.0a) <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- S02** ACM / IEEE-CS / AAAI, Computer Science Curricula 2023, final report <https://csed.acm.org/final-report/>
- S25** W3C, Web Content Accessibility Guidelines 2.2 <https://www.w3.org/TR/WCAG22/>
- S46** ISO/IEC 25010:2023, SQuaRE product quality model <https://www.iso.org/standard/78176.html>

Terms used this week

Requirement

A statement of what the service must do or keep true, written so that an example can show whether it holds. R-01 to R-05 and Q-01 are the ones of this course.

Behavior (functional requirement)

What the service does under a condition: accept a 30-minute booking, reject a 121-minute one, keep a booking confirmed when the message fails.

Quality requirement

How well it does it, with observable acceptance: a threshold with workload and boundary (Q-01), or a categorical outcome such as keyboard completion (R-05).

Business rule

A rule the organization sets about who may do what: an authenticated requester cancels only their own future booking (R-03).

Implementation decision

A way of building it, such as a slider or a relational database. Recorded in the design, next to the requirement it serves; not a requirement.

Elicitation

Finding out the situation behind a request: asking what happened, watching a walk-through, separating what was stated from what was inferred.

Scope

The boundary of a release: who and what is inside. What is outside is deferred with a decision, or omitted by accident.

Assumption

Something inferred rather than stated, with an author. It stays in the register until a stakeholder turns it into a fact or a decision.

Acceptance example

A concrete case with its expected outcome, such as 120 minutes accepted; it is how a requirement is checked and how a change is seen.

Traceability

The links from a requirement to its design decision, its examples and its checks, kept inspectable and current.

Verification

Whether the system matches its written description.

Validation

Whether the description matches the need.

Priority

A reasoned order under current constraints, explained through benefit, dependency, uncertainty, consequence and effort; it changes when the constraints do.